

SPARSE MODULAR REDUCTION WITH LOGARITHMIC FEEDBACK DEPTH

AUTHOR NAME

ABSTRACT. Polynomial modular reduction is a basic operation in binary finite-field arithmetic and in computational-algebra workloads based on repeated Frobenius powers. For a sparse modulus, conventional top-down folding retains sparse shift/XOR work, but its dependency chain can be long when a nonleading term is close to the leading term. We study a different schedule obtained by expressing the high-part recurrence as the inverse of a nilpotent feedback operator. In characteristic two, this inverse factors into Frobenius powers whose individual supports remain controlled by the original tap set. The result is a matrix-free method, which we call *Frobenius-factorized reduction* (FFR), with an exact feedback-stage count and a scheduled-work expression governed by the complete tap geometry.

For a nonleading support of size s , we prove an exact logarithmic feedback depth and an exact active-tap work formula. The former is $r = \lceil \log_2(m/\Delta_{\min}) \rceil$; the latter is bounded by $O(ms(1 + \log(m/s)))$. The reusable schedule and scalar temporary space are also explicit. A matched portable-C evaluation over 1,096 controlled supports and degrees 128 through 131072 identifies distinct FFR, López–Dahab, and gf2x-backed Barrett regions. On the hostile $(h, \Delta_{\min}) = (9, 1)$ slice, Barrett takes between 2.16 and 37.71 times the FFR reduction time, whereas Barrett or López–Dahab wins as support weight or feedback distance grows. These are single-platform reduction-only results; we make no end-to-end computational-algebra performance claim.

1. INTRODUCTION

Arithmetic in a polynomial-basis representation of a binary extension field reduces products and squares modulo a monic polynomial. The same operation appears in polynomial factorization, irreducibility testing, and searches for sparse irreducible or primitive polynomials. In these settings the modulus is public and often fixed for many reductions, but the useful degree, support, and setup-amortization regimes vary substantially.

In the regime studied here, three modulus parameters provide the principal coordinates for reduction time:

- (1) the degree m , which fixes the operand length and hence the amount of word-level data processed;
- (2) the Hamming weight h , including the leading term, which determines the number of nonleading taps available to sparse shift/XOR methods; and

2020 *Mathematics Subject Classification.* Primary 11T06; Secondary 12Y05, 68W10, 68W40.
Key words and phrases. binary polynomials, modular reduction, finite fields, sparse polynomials, Frobenius map, parallel algorithms.

- (3) the nearest-tap distance $\Delta_{\min}$, namely the gap from degree m to the highest positive nonleading tap, when one exists, which controls how far feedback can propagate before leaving the degree- m state; without such a tap there is no feedback chain.

These three parameters do not determine runtime by themselves: complete tap placement, word alignment, setup, and the multiplication backend still affect the observed constants. They nevertheless separate the main size, sparsity, and feedback effects and therefore define the axes and slices used in our evaluation.

When the nonleading part is sparse, top-down folding uses little setup and can be extremely effective. Its work and latency, however, are controlled by different geometric features. Hamming weight controls how many shifts and XORs one fold introduces, whereas the distance from the leading term to the nearest internal tap controls how far feedback can propagate. Equal-degree, equal-weight moduli may therefore induce very different dependency chains.

Barrett and Montgomery reduction avoid this particular serial recurrence, but introduce multiplication kernels, modulus-dependent constants, and different setup costs. Reciprocal methods and dense linear maps expose additional parallelism, but do not automatically preserve sparse support. We ask whether the sparse feedback chain can be shortened without first materializing a dense reciprocal or reduction matrix.

For an input of degree below twice the modulus degree, reduction gives a finite forced recurrence. We write it as $(I + U)Y = H$ for a truncated shift operator U . When $U \neq 0$, characteristic two gives

$$(1) \quad (I + U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}), \quad r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil,$$

and $U^{2^r} = 0$; when $U = 0$, no feedback stage is needed. Each Frobenius power doubles the original tap distances. Mixed feedback paths arise through composition across stages rather than by expanding a dense reciprocal inside one stage.

The contributions are: (i) a nilpotent-feedback formulation for any monic binary modulus; (ii) a sparse characteristic-two factorization; (iii) exact coefficient- and packed-word work-feedback-depth geometry, including schedule and temporary-space bounds; and (iv) a reproducible portable-C study that maps the measured winning and losing regions against serial folding, López-Dahab reduction, and gf2x-backed Barrett reduction.

Correctness does not require irreducibility; sparsity is a performance condition rather than an applicability condition. We do not identify feedback depth with Boolean gate depth, claim an unconditional improvement to complete field multiplication, or infer a side-channel result from a public fixed schedule alone.

Sections 2–3 fix the problem and comparison models. Sections 4–6 develop the method and analysis. Section 7 states the prior-art boundary. Sections 8 and 9 contain the implementation and application evidence, and Section 10 concludes.

2. PRELIMINARIES AND COST MODELS

Let

$$(2) \quad g(x) = x^m + q(x) = x^m + \bigoplus_{t \in T} x^t \in \mathbb{F}_2[x], \quad T \subseteq \{0, \dots, m-1\},$$

be monic, and set $h = 1 + |T|$. For $t \in T$, define $\Delta_t = m - t$. If $T \neq \emptyset$, let

$$(3) \quad \Delta_{\min} = \min\{\Delta_t : t \in T\}.$$

When $T \subseteq \{0\}$, the feedback operator below is zero and we set $r = 0$; otherwise r is defined by (9).

Identify degree-below- m polynomials with vectors in $\mathbb{F}_2^m$. Let N be the truncated right shift

$$(4) \quad (N^d y)_i = \begin{cases} y_{i+d}, & 0 \leq i < m-d, \\ 0, & m-d \leq i < m, \end{cases} \quad N^m = 0.$$

For $A \in \mathbb{F}_2[x]$ with $\deg A < 2m$, write

$$(5) \quad A = L + x^m H, \quad \deg L, \deg H < m.$$

For degree-below- m input Y , define linear maps V and U by

$$(6) \quad \begin{aligned} qY &= V(Y) + x^m U(Y), \\ V(Y) &= \sum_{t \in T} (x^t Y \bmod x^m), & U &= \sum_{t \in T} N^{\Delta_t}. \end{aligned}$$

Here and below an empty sum denotes the zero map. The term $t = 0$, when present, contributes the identity to V and the zero shift N^m to U . Monicity is sufficient for a unique remainder; irreducibility is required only when the quotient is intended to be a field.

We distinguish scheduled coefficient work, machine-word work, feedback depth, bounded-fan-in gate depth, setup, and temporary space. Scheduled coefficient work is neither an instruction count nor a minimal XOR-circuit count. Machine latency and throughput are measured under a fixed platform and timed boundary. For the packed implementation, coefficient i is bit $i \bmod W$ of word $\lfloor i/W \rfloor$ in a little-endian array of $n = \lceil m/W \rceil$ words; padding above coefficient $m - 1$ is masked on state extraction and output. Caller-owned inputs and outputs are excluded from temporary-space counts. Section 8 separately fixes the steady-state reduction boundary, reusable-plan setup, storage, and any amortization over a stated reuse count.

3. EXISTING REDUCTION PARADIGMS

Serial folding repeatedly eliminates high terms through the sparse support. It has low setup, but a tap at distance $\Delta_{\min}$ can create on the order of $m/\Delta_{\min}$ dependent folds. Fixed trinomial and pentanomial formulae, generated code, and López-Dahab reduction can be stronger under their family assumptions [9].

Dense linear maps and generic parallel division expose a different tradeoff. Polynomial division can be reduced to triangular Toeplitz or reciprocal inversion [1]. Barrett and Montgomery instead use polynomial products and modulus-dependent constants. Sparse shift/XOR counts do not determine their machine cost.

Generated fixed-modulus code is outside the general-code winner contract. Montgomery is also excluded: REDC maps A to $AR^{-1} \bmod g$, rather than performing the frozen direct reduction $A \mapsto A \bmod g$.

TABLE 1. Frozen direct-reduction comparison contract. “Primary” denotes a winner-eligible portable-C method; “diagnostic” denotes a screened baseline.

Method	Applicability and evidence role	Reusable plan and principal boundary
FFR	Arbitrary monic modulus; primary method.	Doubled-shift schedule and one state plus a sentinel; cost depends on tap geometry and word traffic.
Serial	Arbitrary monic modulus; screened sparse baseline.	Tap descriptors and two states; small $\Delta_{\min}$ creates a long dependency chain.
BarrettGF2X	Arbitrary monic modulus; primary multiplication-based baseline.	Reciprocal and reusable product buffers; cost depends on gf2x multiplication thresholds.
López–Dahab	$\deg q \leq m - W$; primary baseline where applicable.	Tap descriptors and a $2m$ -bit work buffer; unavailable outside its degree condition.
Dense map	Fixed modulus with packed map at most 64 MiB; diagnostic baseline.	Packed $m \times m$ map and high-part scratch; storage is quadratic.

4. SPARSE FEEDBACK OPERATORS

Lemma 4.1 (Low/high decomposition). *For (5) and (6),*

$$(7) \quad A - gY = L + V(Y) + x^m(H + Y + U(Y)).$$

Consequently, if $H = (I + U)Y$, then $A \bmod g = L + V(Y)$.

Proof. Using (5) and (6), and using that subtraction and addition coincide over $\mathbb{F}_2$, we obtain

$$\begin{aligned} A - gY &= L + x^m H + x^m Y + qY \\ &= L + V(Y) + x^m(H + Y + U(Y)). \end{aligned}$$

If $H = (I + U)Y$, the entire high-part factor in the last display vanishes. Hence $A - (L + V(Y)) = gY$, while $\deg(L + V(Y)) < m$. The uniqueness of the remainder on division by the monic polynomial g proves the claim. $\square$

Lemma 4.2 (Sparse Frobenius powers). *For every $k \geq 0$,*

$$(8) \quad U^{2^k} = \sum_{t \in T} N^{2^k \Delta_t},$$

where shifts by at least m are zero.

Proof. The truncated shifts satisfy $N^a N^b = N^{a+b}$ and therefore commute. In the endomorphism algebra of $\mathbb{F}_2^m$, commuting elements satisfy $(X_1 + \cdots + X_s)^2 =$

$X_1^2 + \dots + X_s^2$. Starting from $U = \sum_{t \in T} N^{\Delta_t}$ and iterating this identity k times gives

$$U^{2^k} = \sum_{t \in T} (N^{\Delta_t})^{2^k} = \sum_{t \in T} N^{2^k \Delta_t}.$$

Finally, $N^d = 0$ for $d \geq m$. For $d < m$, the matrix of N^d occupies the d th superdiagonal. Distinct surviving shifts therefore have disjoint matrix support and cannot cancel. $\square$

Theorem 4.3 (Sparse feedback inverse). *If $U = 0$, set $r = 0$; then $(I + U)^{-1} = I$, with the empty product understood as I . Otherwise, for*

$$(9) \quad r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil,$$

we have $U^{2^r} = 0$ and

$$(10) \quad (I + U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}).$$

Moreover, r is the least nonnegative integer for which $U^{2^r} = 0$.

Proof. Suppose $U \neq 0$. Then T contains a positive tap, $\Delta_{\min} < m$, and $r \geq 1$. By the definition of r , every $t \in T$ satisfies $2^r \Delta_t \geq 2^r \Delta_{\min} \geq m$. Lemma 4.2 therefore gives $U^{2^r} = 0$. On the other hand, $2^{r-1} \Delta_{\min} < m$, so the unique tap at distance $\Delta_{\min}$ contributes the nonzero shift $N^{2^{r-1} \Delta_{\min}}$ to $U^{2^{r-1}}$. Thus r is minimal.

For any $j \geq 0$, repeated use of $(I + X)^2 = I + X^2$ gives the telescoping identity

$$(I + U) \prod_{k=0}^{j-1} (I + U^{2^k}) = I + U^{2^j}.$$

Taking $j = r$ makes the right-hand side I . All factors are polynomials in U and hence commute, so the product is both a left and a right inverse of $I + U$. The case $U = 0$ is immediate. $\square$

This is a factored truncated reciprocal. Formal-series inversion itself is classical [5]; the proposed contribution is the sparse fixed-length realization and its complete tap-geometry analysis.

5. FROBENIUS-FACTORIZED REDUCTION

We call the resulting method *Frobenius-factorized reduction* (FFR), because it applies the Frobenius factors of the truncated feedback inverse without materializing that inverse.

For every $k \geq 0$, define

$$(11) \quad T_k = \{t \in T : 2^k \Delta_t < m\}.$$

Then $T_k = \emptyset$ for $k \geq r$. The public schedule stores shifts $2^k \Delta_t$ for $t \in T_k$, $0 \leq k < r$, and the taps for final low assembly.

Algorithm 1 (Frobenius-factorized reduction). Split $A = L + x^m H$ and set $Y \leftarrow H$. For $k = 0, \dots, r-1$, read one immutable stage input Y_{old} and compute synchronously

$$Y_{\text{new}} = Y_{\text{old}} + \sum_{t \in T_k} N^{2^k \Delta_t} Y_{\text{old}}.$$

Set $Y \leftarrow Y_{\text{new}}$ and return $L + V(Y)$.

An in-place tap loop that exposes updates within a stage computes a different recurrence.

Theorem 5.1 (Correctness). *For every monic $g \in \mathbb{F}_2[x]$ of degree m and every A with $\deg A < 2m$, Algorithm 1 returns $A \bmod g$.*

Proof. At stage k , every summand reads the same value Y_{old} , so the stage maps its input to

$$\left(I + \sum_{t \in T_k} N^{2^k \Delta_t} \right) Y_{\text{old}} = (I + U^{2^k}) Y_{\text{old}}$$

by Lemma 4.2. After all stages,

$$Y = \prod_{k=0}^{r-1} (I + U^{2^k}) H = (I + U)^{-1} H$$

by Theorem 4.3; the order of the factors is immaterial because they commute. Hence $H = (I + U)Y$, and Lemma 4.1 shows that the returned polynomial $L + V(Y)$ is $A \bmod g$. $\square$

Example 5.2. Let

$$g = x^{16} + x^{15} + x^{11} + x^3 + 1.$$

The nonconstant taps have distances 1, 5, and 13; the constant tap has distance 16 and contributes no feedback. Thus $r = 4$, and Algorithm 1 applies the four maps

$$I + N + N^5 + N^{13}, \quad I + N^2 + N^{10}, \quad I + N^4, \quad I + N^8.$$

The shifts 26, 20, and 52 have disappeared by truncation. Their mixed feedback paths have not been discarded: they arise implicitly when these four factors are composed. For the resulting state Y , the algorithm returns

$$L + Y + (x^3 Y \bmod x^{16}) + (x^{11} Y \bmod x^{16}) + (x^{15} Y \bmod x^{16}).$$

Remark 5.3 (Boundary cases). If $T = \emptyset$, then $g = x^m$, $U = V = 0$, no feedback stage is executed, and the remainder is L . If $T = \{0\}$, then $g = x^m + 1$, $U = 0$ and $V = I$, so the remainder is $L + H$. Omitting the constant tap merely removes the identity summand from V ; it does not change the argument. Dense and reducible moduli satisfy the same correctness theorem, although density can make the schedule unattractive. Finally, a degree m not divisible by the machine-word width changes masking and storage in an implementation, not the algebraic algorithm.

6. WORK-DEPTH-SPACE GEOMETRY

We first count *scheduled coefficient contributions*: adding one shifted source coefficient to one destination coefficient costs one unit. Retaining the unshifted stage input, copying a vector, testing a public bound, and reading a schedule descriptor are not included in this measure. Let $s = |T|$, let $h_k = |T_k|$, and define

$$(12) \quad \ell_t = \#\{k \geq 0 : 2^k \Delta_t < m\} = \left\lceil \log_2 \frac{m}{\Delta_t} \right\rceil.$$

The equality includes the constant tap: if $t = 0$, then $\Delta_t = m$ and $\ell_t = 0$.

Theorem 6.1 (Geometry-sensitive scheduled work). *The number of stored feedback shifts and the feedback contribution work are, respectively,*

$$(13) \quad S_{\text{fb}} = \sum_{k=0}^{r-1} h_k = \sum_{t \in T} \ell_t,$$

$$(14) \quad W_{\text{fb}} = \sum_{t \in T} \sum_{k \geq 0} [m - 2^k \Delta_t]_+, \quad [z]_+ = \max\{z, 0\}.$$

The final low-part assembly has scheduled coefficient work

$$(15) \quad W_V = \sum_{t \in T} (m - t) = \sum_{t \in T} \Delta_t.$$

If $s \geq 1$, then

$$(16) \quad S_{\text{fb}} \leq s + \log_2 \frac{m^s}{s!} \leq s \left(1 + \log_2 \frac{em}{s} \right),$$

$$(17) \quad W_{\text{fb}} \leq m S_{\text{fb}} = O\left(ms \left(1 + \log \frac{m}{s} \right) \right),$$

$$(18) \quad W_{\text{fb}} + W_V = O\left(ms \left(1 + \log \frac{m}{s} \right) \right).$$

The inequality in (17) is strict whenever $U \neq 0$.

Proof. Tap t appears precisely for the ℓ_t integers k satisfying $2^k \Delta_t < m$, proving (13). At such an occurrence, the right shift affects exactly $m - 2^k \Delta_t$ coefficient positions. Summing these contributions proves (14). Similarly, $x^t Y \bmod x^m$ contains $m - t = \Delta_t$ scheduled source positions, which proves (15).

Order the distinct distances as $1 \leq d_1 < \dots < d_s \leq m$. Since $d_i \geq i$ and $[z] \leq z + 1$,

$$S_{\text{fb}} \leq \sum_{i=1}^s \left(1 + \log_2 \frac{m}{i} \right) = s + \log_2 \frac{m^s}{s!}.$$

The standard inequality $s! \geq (s/e)^s$ gives the second part of (16). Every one of the S_{fb} feedback shifts affects at most m positions, and affects strictly fewer than m positions when it is nonzero. This proves (17), including the non-strict boundary $W_{\text{fb}} = S_{\text{fb}} = 0$ for a constant-only tap set. Finally, $W_V \leq ms$, and (18) follows. $\square$

Corollary 6.2 (Feedback depth and a gate-depth upper bound). *If $U \neq 0$, Algorithm 1 has exactly*

$$r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil$$

sequential feedback stages; if $U = 0$, it has none. Thus, for fixed m and $\Delta_{\min}$, feedback depth is independent of s . In a fan-in-two XOR model with free fanout and unrestricted temporary wires, the complete reduction has the upper bound

$$(19) \quad D_{\oplus} \leq \sum_{k=0}^{r-1} \lceil \log_2(1 + h_k) \rceil + \lceil \log_2(1 + s) \rceil.$$

Proof. The exact feedback-stage count is the minimality statement in Theorem 4.3. Within stage k , each output coefficient is the sum of its old value and at most h_k shifted values, so a balanced XOR tree has depth at most $\lceil \log_2(1 + h_k) \rceil$. The stages are composed sequentially. Final assembly adds L and at most s shifted values, giving the last term in (19). $\square$

Equation (19) is an upper bound in a specified circuit model, not a lower bound and not a hardware-depth claim. In particular, feedback depth treats a whole synchronous vector update as one stage, whereas gate depth accounts for the XOR tree inside that stage.

6.1. Packed-word geometry. Let W be the machine-word width. Put $n = \lceil m/W \rceil$. The following counts describe destination-word contributions; they are not instruction counts. A feedback shift by $d < m$ has potentially nonzero coefficients in $\lceil (m-d)/W \rceil$ destination words. A low-assembly shift by t touches

$$n - \left\lfloor \frac{t}{W} \right\rfloor$$

destination words. The latter is not always $\lceil (m-t)/W \rceil$: a non-word-aligned interval may intersect an additional word.

Proposition 6.3 (Word-level loop geometry). *For $U \neq 0$, set*

$$a_k = \left\lceil \frac{m - 2^k \Delta_{\min}}{W} \right\rceil \quad (0 \leq k < r).$$

The useful feedback-word support and the shifted-word contributions traversed by the scalar stage loops are, respectively,

$$(20) \quad \hat{B}_{\text{fb}} = \sum_{t \in T} \sum_{\substack{k \geq 0 \\ 2^k \Delta_t < m}} \left\lceil \frac{m - 2^k \Delta_t}{W} \right\rceil,$$

$$(21) \quad B_{\text{fb}}^{\text{loop}} = \sum_{k=0}^{r-1} \sum_{t \in T_k} \min \left\{ a_k, n - \left\lfloor \frac{2^k \Delta_t}{W} \right\rfloor \right\},$$

$$(22) \quad B_V = \sum_{t \in T} \left(n - \left\lfloor \frac{t}{W} \right\rfloor \right).$$

They satisfy

$$(23) \quad \hat{B}_{\text{fb}} \leq B_{\text{fb}}^{\text{loop}} \leq \hat{B}_{\text{fb}} + S_{\text{fb}}.$$

An in-place scalar implementation makes one n -word high-part extraction, traverses a_k destination words at feedback stage k , and makes one n -word final assembly pass, in addition to the $B_{\text{fb}}^{\text{loop}} + B_V$ shifted contributions.

Proof. A right shift by d occupies coefficient positions $0, \dots, m-d-1$, which span $\lceil (m-d)/W \rceil$ words. A left shift by t occupies positions $t, \dots, m-1$, which span the words numbered $\lfloor t/W \rfloor, \dots, n-1$. At stage k , the smallest active shift is $2^k \Delta_{\min}$, so the union of all affected destinations spans exactly a_k words. For a particular feedback shift $d = 2^k \Delta_t$, the scalar loop also requires the source-word index $\lfloor d/W \rfloor$ to remain below the end of the state. It therefore visits

$$\min\{a_k, n - \lfloor d/W \rfloor\}$$

destinations. This is either the useful count $\lceil (m-d)/W \rceil$ or that count plus one; the latter case performs only a padding-word contribution. Summing proves (23). The extraction and assembly statements follow from traversing the m -bit input and output states once. $\square$

A word-aligned contribution can be formed from one source word. A non-word-aligned contribution can require two adjacent source words, two machine shifts, and an XOR before accumulation. Consequently, Proposition 6.3 records loop geometry but deliberately does not convert aligned and cross-word contributions into an ISA-independent latency estimate.

Proposition 6.4 (Temporary space and schedule setup). *Excluding the caller-owned input and output, the in-place scalar reduction can be performed with one m -bit state. The implementation evaluated in this paper allocates exactly $n + 1$ plan-owned scratch-array words: n state words and one zero sentinel. An explicit reusable plan stores*

r stage records, S_{fb} feedback-shift records, s assembly-shift records,

in addition to constant-size metadata. Such a plan can be constructed in $O(s + r + S_{\text{fb}})$ word-RAM operations and the same asymptotic number of records.

Proof. For a positive right shift, output coefficient i depends only on old coefficients with indices at least i . Updating coefficients from low to high therefore never destroys a value needed by a later destination. This permits one in-place state; the extra sentinel is an implementation device for branch-free adjacent-word reads.

The record counts follow directly from (13). For the setup bound, compute every ℓ_t by repeated doubling, at total cost $O(s + S_{\text{fb}})$. Because each tap is active on the stage interval $0, \dots, \ell_t - 1$, a difference array followed by one prefix sum computes all r per-stage counts in $O(s + r)$ operations. A second pass emits the ℓ_t doubled shifts of each tap into the corresponding stage buckets in $O(S_{\text{fb}})$ operations. $\square$

The setup bound describes a two-pass schedule construction, not compilation of generated code. The experiments separately time the concrete reusable plan builders. For comparison, serial sparse folding needs no doubled-shift schedule but, when $U \neq 0$, can execute up to $\lceil m/\Delta_{\min} \rceil$ dependent feedback iterations; López-Dahab has $\Theta(n(1 + s))$ shift/scatter work under its $\deg q \leq m - W$ applicability condition; the Barrett baseline performs two polynomial multiplications on $\Theta(n)$ -word operands after reciprocal setup; and a materialized dense map stores $\Theta(mn)$ words and performs $\Theta(mn)$ row-word products. These are different cost primitives, so their constants and crossovers are determined experimentally rather than by equating the displayed source-level counts.

For a compact comparison, let $M(n)$ and $D_M(n)$ denote the work and dependency depth of an n -word polynomial multiplication. For $U \neq 0$, let $\nu_U = \min\{j \geq 1 : U^j = 0\}$; then $\nu_U \leq \lceil m/\Delta_{\min} \rceil$. The boundary $U = 0$ has no dependent feedback fold. Table 2 collects the resulting method-specific costs. The formulas describe the implementations or operation models used here, not compiler instructions or measured cycles.

6.2. Uniform fixed-weight supports. We next record what the deterministic geometry implies for one explicit distribution. Fix m and $1 \leq s \leq m$, and choose T uniformly among the s -subsets of $\{0, \dots, m - 1\}$. Equivalently, the distances form a uniform s -subset of $\{1, \dots, m\}$. This model permits a zero constant coefficient and reducible moduli; it is not a distribution on irreducible polynomials.

TABLE 2. Reduction work and dependency depth. Here $n = \lceil m/W \rceil$, $s = |T|$, and fan-in-two depth assumes free fanout. The serial row assumes $U \neq 0$, and the López–Dahab row assumes $\deg q \leq m - W$.

Method	Work	Depth
FFR	$C_{\text{coeff}} = W_{\text{fb}} + W_V,$	$D_{\text{fb}} = r = \left\lceil \log_2 \frac{m}{\Delta_{\min}} \right\rceil,$
	$C_{\text{word}} = O\left(n + \sum_k a_k + B_{\text{fb}}^{\text{loop}} + B_V\right)$	$D_{\oplus} \leq \sum_{k=0}^{r-1} [\log_2(1 + h_k)] + \lceil \log_2(1 + s) \rceil$
Serial	$C_{\text{word}} = O(sn\nu_U)$	$D_{\text{fb}} \leq \nu_U \leq \left\lceil \frac{m}{\Delta_{\min}} \right\rceil,$ $D_{\oplus} = O(\nu_U \log(s + 1))$
López–Dahab	$C_{\text{word}} = \Theta(n(1 + s))$	$D_{\text{loop}} = n + O(1),$ $D_{\oplus} = O(n \log(s + 1))$
BarrettGF2X	$C_{\text{word}} = 2M(n) + O(n)$	$D = 2D_M(n) + O(1)$
Dense map	$C_{\text{word}} = \Theta(mn) = \Theta(m^2/W)$	$D_{\oplus} = O(\log m)$

Theorem 6.5 (Random-support schedule geometry). *Under the uniform fixed-weight model,*

$$(24) \quad \mathbb{E}[h_k] = \frac{s}{m} \left\lfloor \frac{m-1}{2^k} \right\rfloor < \frac{s}{2^k},$$

$$(25) \quad \mathbb{E}[S_{\text{fb}}] < 2s,$$

$$(26) \quad \mathbb{E}[r] \leq \lceil \log_2 s \rceil + 2,$$

$$(27) \quad \mathbb{E}[W_{\text{fb}}] < 2ms.$$

Moreover, $\mathbb{E}[W_V] = s(m+1)/2$, and hence the expected total scheduled coefficient contribution work is $O(ms)$.

Proof. There are exactly $\lfloor (m-1)/2^k \rfloor$ distances $d \in \{1, \dots, m\}$ with $2^k d < m$. Each distance belongs to the random support with probability s/m , proving (24). Summing over $k \geq 0$ and using $S_{\text{fb}} = \sum_k h_k$ gives (25).

Since $r = \sum_{k \geq 0} \mathbf{1}_{\{h_k > 0\}}$, Markov's inequality and (24) give

$$\Pr(h_k > 0) \leq \min\{1, s/2^k\}.$$

Summing the first $\lceil \log_2 s \rceil$ unit bounds and the remaining geometric tail proves (26). The deterministic inequality $W_{\text{fb}} \leq mS_{\text{fb}}$ proves (27). Finally, every distance has inclusion probability s/m , so

$$\mathbb{E}[W_V] = \frac{s}{m} \sum_{d=1}^m d = \frac{s(m+1)}{2}.$$

□

Exact work depends on every tap distance, not only on $(h, \Delta_{\min})$. The bounds above therefore describe families and schedules; they do not claim that one method wins at every modulus with the same degree and weight.

7. PRIOR ART AND NOVELTY BOUNDARY

Reciprocal and triangular Toeplitz inversion for polynomial division are classical [1], as are formal-power-series inversion [5]. Parallel triangular Toeplitz and recurrence solvers already use doubling, bandwidth, or prefix structure [2, 12, 8, 6]. Consequently, neither the finite geometric inverse nor logarithmic-depth recurrence solving is a novelty claim here. Those general methods may use dense reciprocals, polynomial multiplication, or expanded state; our question is whether the particular sparse fixed-state realization retains useful support geometry.

The closest local antecedent is the term-preserving look-ahead transformation for LFSRs: in characteristic two, replacing a generator by its square doubles the feedback distances without increasing its term count [10]. Parallel LFSR and CRC architectures also construct equivalent state-space updates for arbitrary generator polynomials [3]. These works preclude a claim that term-preserving Frobenius doubling or parallel feedback is new. They transform or parallelize a homogeneous recurrence, whereas Algorithm 1 applies successive factors of an implicit inverse to a forced length- m triangular system without enlarging the state.

Sparse binary-field reduction includes word- and bit-parallel polynomial-basis methods [15] and strong specialized formulae [9]. Niehues, Custodio, and Panario give a uniform top-down reducer for arbitrary low-weight moduli and explicitly observe that tap placement can induce a long critical path [13]. Finite-field multiplier architectures additionally use multi-degree reduction, subexpression sharing, and balanced XOR trees [11]. FFR therefore claims neither the first arbitrary-tap reducer nor the first parallel reduction circuit; its measured comparison keeps serial folding and applicable López–Dahab reduction as direct baselines.

Barrett and Montgomery provide multiplication-based alternatives, including restricted precomputation-free families [7]. Thus “without precomputation” is not a novelty claim either. Our plans avoid a materialized reciprocal or dense matrix, but still have a separately measured setup cost.

Repeated Frobenius powering in factorization gives the closest application context. Von zur Gathen and Gerhard combine theory, implementation, storage, and complete factorization over $\mathbb{F}_2$ [4]; irreducible polynomial construction gives another context [14].

Our narrow boundary is the combination of arbitrary monic-modulus correctness, sparse support-sensitive factors, logarithmic feedback depth, no materialized dense reciprocal or matrix, and exact geometry from all tap distances. We do not claim the first reciprocal, Frobenius identity, parallel recurrence solver, or balanced XOR realization.

8. IMPLEMENTATIONS AND EVALUATION

8.1. Implementations and validation. The native interface exposes FFR, serial top-down folding, and gf2x-backed Barrett reduction through the same operation

$$A \longmapsto A \bmod g, \quad \deg A < 2m.$$

FFR and serial folding use the same packed-word shift/XOR primitive. The frozen implementation and dataset retain **GS** as their internal method identifier. We also implemented the ordinary-loop López–Dahab reducer, using it only when $\Delta_{\min} \geq W$, and a packed dense linear map for diagnostic screening. An independent long-division implementation supplies the correctness oracle but is not a performance method. Montgomery REDC is excluded because it computes $AR^{-1} \bmod g$ in a different representation and hence does not satisfy the frozen direct-reduction contract.

Before timing, the binary implementations are compared with independent long division at word boundaries and on deterministic random instances. Separate falsification suites test the algebraic identities over $\mathbb{F}_2$, $\mathbb{F}_4$, and $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$, together with the odd-characteristic sign convention over $\mathbb{F}_3$. These checks can expose implementation or statement errors, but are not proofs of Theorem 5.1.

8.2. Experimental contract. We measured portable scalar C on one recorded x86-64 host, pinned to logical CPU 0. The programs were compiled with GCC 16.1.1 using `-O3 -std=c11 -Wall -Wextra`; Barrett used `/usr/lib/libg2x.so.3.0.0`. The governor, energy-performance preference, and turbo state accompany every retained row. Thus every timing conclusion below is platform-specific.

For each support, the input corpus consists of eight materialized polynomials $A = L + x^m H$, where L and H are independent uniform m -bit polynomials. One complete warm-up invocation is discarded. A retained trial uses twelve batch repetitions, with the method order cyclically rotated so that each enabled method occupies every timing position equally often. The setup, input generation, allocation, correctness checks, and output handling lie outside the timed call to `reduce_into`. No observation is deleted or winsorized. The data contain either 31 or 127 complete trials per support, according to the frozen collection cohort.

The controlled corpus contains 1,096 constant-free supports at

$$m \in \{128, 512, 2048, 8192, 32768, 131072\}.$$

It varies h and $\Delta_{\min}$ independently: the nearest tap is fixed at $m - \Delta_{\min}$, and the remaining taps are spread deterministically. Every point compares FFR with Barrett. Serial is present in the 392 points retained from the initial and López–Dahab screening grids, while high-weight extensions omit it; López–Dahab is present precisely at measured points satisfying $\Delta_{\min} \geq 64$. Method availability is therefore part of each point, and an unmeasured method is never treated as a loser.

For each method at each support, we take the median across trials and compute a deterministic 10,000-resample bootstrap 95% interval. A unique winner is assigned only if every participating median has relative interval half-width at most 1%, the fastest median is at least 1% below every competitor, and every paired fastest/competitor ratio interval lies strictly below one. Failures of these tests are retained as timing instability, operational ties, or statistical ties rather than forced into a phase boundary.

8.3. Measured winner regions. Figure 1 gives the complete measured phase diagram. Its axes are the two principal geometric parameters rather than ordinal grid indices. Every block is one observed support; blank regions were not measured and are not interpolated. To expose the predicted regime decomposition, we overlay a fitted cost model. At each fixed m , a deterministic alternating split of the sorted grid fits affine times for FFR from its portable scalar block-work count and for

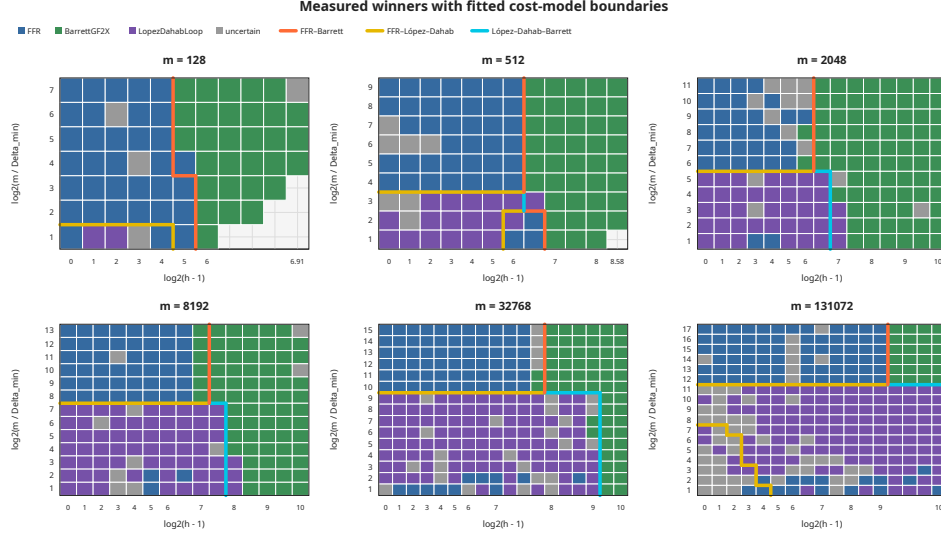


FIGURE 1. Measured winner regions for six fixed degrees. The horizontal axis is $\log_2(h-1)$ and the vertical axis is $\log_2(m/\Delta_{\min})$. Colors identify the unique winner among the methods available at that point; gray cells are uncertain. FFR and Barrett are present everywhere, whereas López–Dahab is admitted only at its tested $\Delta_{\min} \geq 64$ boundary. Thick lines mark pair-specific interfaces predicted by the fixed- m fitted cost model; they neither change measured classifications nor interpolate unmeasured cells.

López–Dahab from $\lceil m/W \rceil(h-1)$; Barrett is represented by its fixed- m calibration median. The minimum predicted time labels each measured coordinate, and a line is drawn only where adjacent predicted labels differ. Thus the blocks remain measurements whereas the lines are descriptive model predictions.

Of the 1,096 cells, 983 have a unique winner: FFR wins 321, Barrett wins 305, and López–Dahab wins 357. The remaining 113 cells comprise 103 timing-unstable cells and 10 operational ties; no cell is classified as a statistical tie. The largest panel, $m = 131072$, deliberately retains only 31 trials and consequently has 52 unstable cells out of 289. This visible uncertainty is part of the result, not evidence for any adjacent region.

The diagram separates three regimes. FFR occupies the sparse, difficult-feedback region. As weight increases, multiplication-based Barrett eventually wins even when $\Delta_{\min}$ is small. When $\Delta_{\min} \geq 64$, López–Dahab increasingly dominates the friendly-feedback region as m grows. Across rows with $\Delta_{\min} < 64$, the observed persistent Barrett onset is $h = 33\text{--}65$, 97 , $65\text{--}97$, 129 , $225\text{--}257$, and 641 for the six successive degrees. In the López–Dahab-applicable rows the corresponding onsets are 65 , 129 , $129\text{--}193$, $257\text{--}385$, $513\text{--}769$, and beyond the largest sampled weight 1025 . These are sampled, platform-specific crossovers, not monotonicity theorems.

These medians use `portable-scalar-c:v1`, `reduction-steady-state:v1`, and the `gf2x:v1` multiplication backend on the recorded primary machine. They establish a platform-specific phase boundary, not a cross-platform ordering.

TABLE 3. Portable-C reduction summary on the controlled constant-free $\Delta_{\min} = 1$ slice. FFR time and the first ratio use $h = 9$. The crossover is the first stable Barrett winner after which every later stable sampled weight is also Barrett; uncertain cells neither create nor break it.

m	FFR (ns)	Barrett/FFR	crossover h	ratio at crossover
128	64.1	2.164	33	0.881
512	272.9	3.807	97	0.718
2048	1079.2	4.247	97	0.664
8192	4390.2	8.654	129	0.819
32768	17008.4	16.768	257	0.881
131072	68167.1	37.712	641	0.886

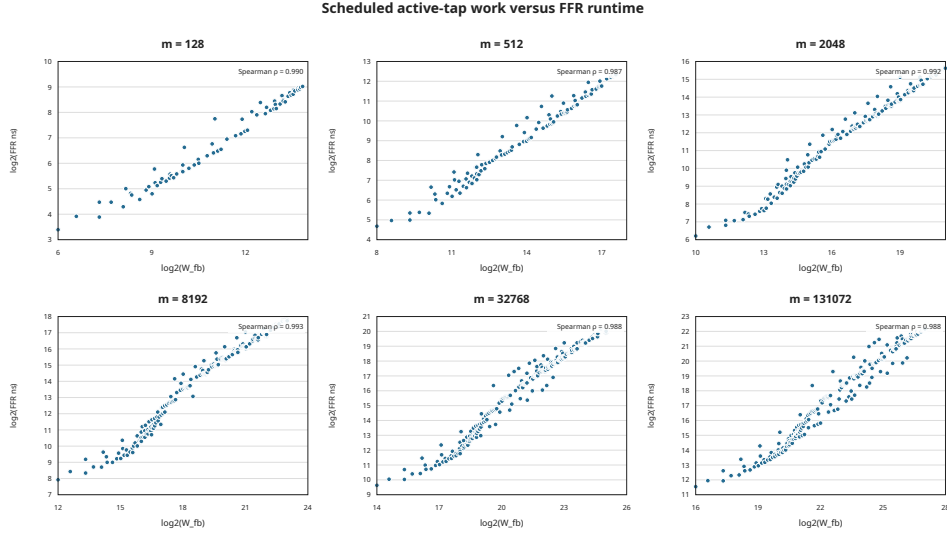


FIGURE 2. Scheduled active-tap work versus median FFR reduction time for all 1,096 controlled supports, shown separately at fixed m . Both axes use base-two logarithms. The displayed ρ is the within-panel Spearman correlation.

8.4. Does the work model predict FFR time? Figure 2 compares the formal scheduled coefficient work W_{fb} with measured FFR time. Within each fixed degree, the Spearman correlation ranges from 0.9868 to 0.9934. Thus the complete tap geometry captured by W_{fb} orders these controlled scalar runtimes unusually well. The residual scatter is not assigned to cache traffic, alignment, or any other hardware cause because no counter evidence was collected, and the correlation is not an instruction-count identity.

8.5. Setup and storage. Setup and requested plan-owned storage follow separately frozen contracts. The storage counts exclude allocator overhead, shared modulus storage, benchmark buffers, and transient library workspace. The setup

TABLE 4. Median setup time and requested plan-owned bytes at $(h, \Delta_{\min}) = (9, 1)$. Each entry is “nanoseconds / bytes.”

m	trials	FFR	Serial	Barrett
128	127	596.5 / 744	119.5 / 416	782.0 / 184
512	127	719.5 / 888	123.5 / 512	4770.5 / 376
2048	127	887.5 / 1176	137.0 / 896	50799.0 / 1144
8192	127	1152.0 / 2040	149.0 / 2432	558369.5 / 4216
32768	127	1312.0 / 5208	235.0 / 8576	7760258.5 / 16504
131072	31	2832.5 / 17592	856.5 / 33152	119030566.0 / 65656

measurements show a tradeoff hidden by steady-state timings: Barrett’s reciprocal construction becomes expensive at large m , while FFR retains a modest schedule-construction cost. This does not alter the reduction-only winner diagram. For an explicitly chosen reuse count K , the artifact separately derives

$$T_{\text{total}}(K) = T_{\text{setup}} + KT_{\text{reduce}};$$

we do not fold an unstated amortization factor into the reported reduction times.

9. COMPUTATIONAL-ALGEBRA CONSEQUENCES AND LIMITATIONS

Distinct-degree factorization, Rabin irreducibility testing, and sparse polynomial search repeatedly compute modular Frobenius powers, so reduction is a genuine component of these workloads. The present experiment deliberately isolates that component. It does not measure square formation, multiplication, GCDs, early exits, or complete factorization and irreducibility-testing runs. In particular, the uniform degree-below- $2m$ inputs used in Section 8 are not distributed like polynomial squares, whose odd coefficients vanish. The measured reduction regions are therefore evidence about reducer selection, not about end-to-end workload speed or the best irreducible modulus for an application.

Several additional boundaries matter. The timing corpus uses controlled supports without a constant term, and their irreducibility is unclassified; correctness requires only monicity, but no field-performance conclusion follows. All timings come from one x86-64 host, one compiler configuration, a 64-bit scalar representation, and one gf2x installation. The largest panel retains 52 timing-unstable cells, and serial folding was omitted from the high-weight extension after its initial screening. López-Dahab participates only for $\Delta_{\min} \geq 64$, while the dense map is diagnostic rather than winner-eligible. Setup and requested plan storage are reported separately from steady-state reduction, so a different plan-reuse distribution can change total cost.

Finally, dense supports, small degrees, cross-word shifts, and specialized fixed-modulus reducers can all make FFR unattractive, while fast polynomial multiplication favors Barrett. The theorems give no lower bound for every fixed modulus, no optimal XOR circuit, no constant-time or side-channel guarantee, and no asymptotic improvement to complete field multiplication.

10. CONCLUSION

We recast reduction by an arbitrary monic binary polynomial as inversion of a nilpotent feedback operator. Characteristic-two Frobenius powers yield a matrix-free schedule with exact feedback depth $\lceil \log_2(m/\Delta_{\min}) \rceil$ and work governed by every tap distance. The analysis also separates coefficient work, packed-word loop geometry, setup, and temporary space.

The portable scalar experiments show that this tradeoff has nontrivial algorithm-selection regions. FFR wins for sparse moduli with difficult feedback, gf2x-backed Barrett takes over as weight grows, and ordinary-loop López-Dahab is often strongest when its feedback-distance condition applies. The formal work measure is strongly monotone with FFR runtime on the controlled corpus, but the measured boundaries retain unstable cells and remain platform-specific. Establishing a computational-algebra speedup requires a separate modular-squaring and end-to-end experiment; it is not implied by the reduction-only results reported here.

APPENDIX A. COEFFICIENT-ALGEBRA EXTENSION

Let R be a commutative characteristic-two algebra and let

$$g = x^m + \sum_{t=0}^{m-1} a_t x^t \in R[x]$$

be monic. Define N as before on R^m , and put

$$U = \sum_{t=0}^{m-1} a_t N^{m-t}, \quad V(Y) = \sum_{t=0}^{m-1} a_t (x^t Y \bmod x^m).$$

Theorem A.1 (Coefficient-algebra factorization). *For every $k \geq 0$,*

$$(28) \quad U^{2^k} = \sum_{t=0}^{m-1} a_t^{2^k} N^{2^k(m-t)}.$$

If $U^{2^r} = 0$, then

$$(I + U)^{-1} = \prod_{k=0}^{r-1} (I + U^{2^k}).$$

Consequently, applying these factors to H and returning $L + V(Y)$ computes $A \bmod g$ for every $A = L + x^m H$ of degree below $2m$.

Proof. The summands $a_t N^{m-t}$ commute. Frobenius is therefore additive on their sum, and iterating it gives (28). The telescoping identity

$$(I + U) \prod_{k=0}^{r-1} (I + U^{2^k}) = I + U^{2^r} = I$$

proves the inverse formula. Finally,

$$A - gY = L + V(Y) + x^m(H + Y + U(Y))$$

in characteristic two, so $H = (I + U)Y$ leaves the unique degree-below- m remainder $L + V(Y)$. $\square$

This extension preserves the formal stage schedule, but not necessarily its actual nonzero support. In a nonreduced algebra a coefficient may satisfy $a_t \neq 0$ and $a_t^{2^k} = 0$; for example, $\varepsilon^2 = 0$ in $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$. Thus the exact binary expression W_{fb} is scheduled work in this setting, while an implementation that removes zero coefficients may do less arithmetic. In odd characteristic p , the corresponding radix- p factorization uses $\sum_{j=0}^{p-1} (-U^{p^k})^j$, and the final remainder is $L - V(Y)$ rather than the characteristic-two expression $L + V(Y)$.

APPENDIX B. VALIDATION AND ADDITIONAL RESULTS

The deterministic algebraic suites use independent polynomial long division or direct finite geometric series as their oracles. Over $\mathbb{F}_2$, 20,000 random cases with $m \leq 128$ and all 299,592 monic-modulus/input pairs for $m \leq 6$ pass. Over $\mathbb{F}_2[\varepsilon]/(\varepsilon^2)$, 20,000 random cases and all 266,304 cases for $m \leq 3$ pass; 13,573 random instances exhibit a nonzero coefficient killed by a Frobenius power, motivating the support qualification above. A further 20,000 random $\mathbb{F}_4$ cases and 10,000 random $\mathbb{F}_3$ cases agree with independent inverse and reduction paths. Exhaustive enumeration of all 262,125 nonempty supports for $2 \leq m \leq 18$ finds no counterexample to the scheduled-work formula or its coarse bound.

The repository entry point `make check` runs these theorem-falsification suites together with native differential, boundary, masking, sanitizer, dense, and López–Dahab checks. The frozen artifact uses these commands:

```
make artifact-cost-model
make artifact-microbenchmark ARTIFACT_CPU=0
make artifact-paper
```

Its 67,576-row external dataset and 26 derived outputs are hash-locked. A detached clean checkout reproduced the 24-output predecessor artifact; the fitted-boundary extension has been rebuilt in the current tree and awaits its post-commit detached audit. Exact seeds, tap manifests, commands, compiler and gf2x paths, binary digests, affinity, and frequency policy accompany every retained row. No secondary-platform or end-to-end result is included in this version.

REFERENCES

- [1] D. Bini and V. Pan, *Fast parallel polynomial division via reduction to triangular Toeplitz matrix inversion and to polynomial inversion modulo a power*, Inform. Process. Lett. **21** (1985), no. 2.
- [2] D. Bini, *Parallel solution of certain Toeplitz linear systems*, SIAM J. Comput. **13** (1984), no. 2, 268–276, <https://doi.org/10.1137/0213019>.
- [3] M. Ayinala and K. K. Parhi, *High-speed parallel architectures for linear feedback shift registers*, IEEE Trans. Signal Process. **59** (2011), no. 9, 4459–4469, <https://doi.org/10.1109/TSP.2011.2159495>.
- [4] J. von zur Gathen and J. Gerhard, *Polynomial factorization over $\mathbb{F}_2$* , Math. Comp. **71** (2002), no. 240, 1677–1698, <https://doi.org/10.1090/S0025-5718-02-01421-7>.
- [5] K. S. K. Kalorkoti, *Inverting polynomials and formal power series*, SIAM J. Comput. **22** (1993), no. 3, <https://doi.org/10.1137/0222037>.
- [6] C.-W. Ho and R. C. T. Lee, *A parallel algorithm for solving sparse triangular systems*, IEEE Trans. Comput. **39** (1990), no. 6, 848–852, <https://doi.org/10.1109/12.53610>.
- [7] M. Knezevic, K. Sakiyama, J. Fan, and I. Verbauwhede, *Modular reduction in $\text{GF}(2^n)$ without pre-computational phase*, WAIFI 2008, LNCS 5130, Springer, 2008, pp. 77–87.
- [8] P. M. Kogge and H. S. Stone, *A parallel algorithm for the efficient solution of a general class of recurrence equations*, IEEE Trans. Comput. **C-22** (1973), no. 8.

- [9] J. López and R. Dahab, *High-speed software multiplication in $\mathbb{F}_2^m$* , Progress in Cryptology—INDOCRYPT 2000, LNCS 1977, Springer, 2000.
- [10] J.-C. Lin, S.-J. Chen, and Y. H. Hu, *Cycle-efficient LFSR implementation on word-based microarchitecture*, IEEE Trans. Comput. **62** (2013), no. 4, 832–836, <https://doi.org/10.1109/TC.2012.14>.
- [11] P. K. Meher, *Systolic and non-systolic scalable modular designs of finite field multipliers for Reed–Solomon codec*, IEEE Trans. Very Large Scale Integr. (VLSI) Syst. **17** (2009), no. 6, 747–757, <https://doi.org/10.1109/TVLSI.2008.2006080>.
- [12] M. Morf, *Doubling algorithms for Toeplitz and related equations*, Proc. IEEE Int. Conf. Acoustics, Speech, and Signal Processing, 1980, pp. 954–959, <https://doi.org/10.1109/ICASSP.1980.1171074>.
- [13] L. Boppo Niehues, R. Custodio, and D. Panario, *Fast modular reduction and squaring in $\text{GF}(2^m)$* , Inform. Process. Lett. **132** (2018), <https://doi.org/10.1016/j.ipl.2017.12.002>.
- [14] V. Shoup, *New algorithms for finding irreducible polynomials over finite fields*, Math. Comp. **54** (1990), no. 189, 435–447, <https://doi.org/10.1090/S0025-5718-1990-0993933-0>.
- [15] H. Wu, *Low complexity bit-parallel finite field arithmetic using polynomial basis*, Cryptographic Hardware and Embedded Systems—CHES 1999, LNCS 1717, Springer, 1999, pp. 280–291.

INSTITUTION AND POSTAL ADDRESS

Email address: `author@example.org`